

Étape 4 : Recouvrement de Données

Objectif : Créer un outil pour déchiffrer les fichiers chiffrés par le ransomware.

Objectifs de l'Étape

À la fin de cette étape, vous devez :

-  Comprendre l'algorithme de chiffrement
 -  Compléter le script Python de déchiffrement
 -  Déchiffrer les 3 fichiers de test fournis
 -  Vérifier l'intégrité des données récupérées
 -  Documenter votre outil
-

Fichiers Fournis

```
Etape4_Recouvrement/
├── README.md                # Ce fichier
├── templates/
│   └── recovery_tool.py    # Template à compléter (80% fait)
├── encrypted_files/
│   ├── document.txt.encrypted # Fichier 1 à déchiffrer
│   ├── rapport.txt.encrypted # Fichier 2 à déchiffrer
│   └── clients.txt.encrypted # Fichier 3 à déchiffrer
└── SOLUTIONS.md           # Solutions complètes
```

Instructions

1. Comprendre l'Algorithme XOR

1.1 Principe du XOR

Le ransomware utilise un chiffrement **XOR simple** :

```
Texte clair XOR Clé = Texte chiffré
Texte chiffré XOR Clé = Texte clair
```

Propriété importante : XOR est réversible !

1.2 Exemple

```
# Chiffrement
texte = b"HELLO"
cle = b"KEY"

# H XOR K = Chiffré[0]
# E XOR E = Chiffré[1]
# L XOR Y = Chiffré[2]
# L XOR K = Chiffré[3] (la clé se répète)
# O XOR E = Chiffré[4]

# Déchiffrement (identique !)
chiffre XOR cle = texte original
```



Question 1 : Pourquoi XOR permet-il de chiffrer ET déchiffrer avec la même opération ?

2. Analyser le Template Python

2.1 Ouvrir le Template

```
cd Etape4_Recouvrement/
cat templates/recovery_tool.py
```

2.2 Identifier les Sections TODO

Le template contient **4 TODO** :

TODO 1 : Définir la clé de chiffrement

```
ENCRYPTION_KEY = b"VOTRE_CLE_ICI" # À remplacer
```

TODO 2 : Compléter la fonction `xor_decrypt()`

```
def xor_decrypt(data: bytes, key: bytes) -> bytes:
    # TODO: Implémenter le XOR
```

TODO 3 : Gérer la lecture/écriture des fichiers

```
def decrypt_file(encrypted_filepath, output_filepath):
    # TODO: Lire, déchiffrer, écrire
```

TODO 4 : Scanner un répertoire (déjà fait en grande partie)

```
def find_encrypted_files(directory):  
    # TODO: Trouver tous les .encrypted
```

3. Compléter le Script

3.1 TODO 1 : Définir la Clé

Remplacez la clé avec celle extraite à l'Étape 2 :

```
ENCRYPTION_KEY = b"<key>" # Votre clé extraite avec GDB
```

Vérification :

```
python3 templates/recovery_tool.py
```

Vous devriez voir :  Clé configurée: 19 bytes

3.2 TODO 2 : Fonction XOR

La fonction `xor_decrypt()` est **déjà presque complète** ! Vérifiez qu'elle ressemble à :

```
def xor_decrypt(data: bytes, key: bytes) -> bytes:  
    decrypted = bytearray()  
    key_len = len(key)  
  
    for i, byte in enumerate(data):  
        # Appliquer XOR entre le byte de data et le byte de key  
        # cycliquement  
        decrypted_byte = byte ^ key[i % key_len]  
        decrypted.append(decrypted_byte)  
  
    return bytes(decrypted)
```

 **Question 2 :** Que fait `i % key_len` ? Pourquoi est-ce nécessaire ?

3.3 TODO 3 : Déchiffrement de Fichier

La fonction `decrypt_file()` est aussi **quasiment complète** :

```
def decrypt_file(encrypted_filepath, output_filepath=None):  
    # Lire le fichier chiffré  
    with open(encrypted_filepath, 'rb') as f:  
        .....  
  
    # Déchiffrer  
    decrypted_data = .....  
  
    # Sauvegarder  
    with open(output_filepath, 'wb') as f:  
        f.....
```

 **Question 3** : Pourquoi utilise-t-on 'rb' et 'wb' (mode binaire) ?

3.4 TODO 4 : Scanner un Répertoire

Cette partie est **déjà codée** :

```
def find_encrypted_files(directory):  
    encrypted_files = []  
    for root, dirs, files in os.walk(directory):  
        for file in files:  
            if file.endswith('.encrypted'):  
                encrypted_files.append(os.path.join(root, file))  
    return encrypted_files
```

4. Tester le Recovery Tool

4.1 Test sur un Seul Fichier

```
cd Etape4_Recouvrement/  
  
# Copier le template dans le répertoire courant  
cp templates/recovery_tool.py .  
  
# Tester sur un fichier  
python3 recovery_tool.py encrypted_files/document.txt.encrypted
```

Vous devriez voir :

```
✓ Clé configurée: 19 bytes  
📖 Lecture de encrypted_files/document.txt.encrypted...  
🔓 Déchiffrement en cours...
```

```
💾 Sauvegarde dans encrypted_files/document.txt...  
✅ Succès: encrypted_files/document.txt
```

4.2 Vérifier le Fichier Déchiffré

```
cat encrypted_files/document.txt
```

Vous devriez voir le contenu original :

```
Ceci est un document confidentiel de test.
```

 **Question 4** : Le fichier est-il correctement déchiffré ?

4.3 Test sur Tous les Fichiers

```
python3 recovery_tool.py encrypted_files/
```

Résultat attendu :

```
📋 3 fichier(s) trouvé(s):  
- encrypted_files/document.txt.encrypted  
- encrypted_files/rapport.txt.encrypted  
- encrypted_files/clients.txt.encrypted  
  
✅ Succès: encrypted_files/document.txt  
✅ Succès: encrypted_files/rapport.txt  
✅ Succès: encrypted_files/clients.txt  
  
✅ Fichiers déchiffrés avec succès: 3
```

4.4 Vérifier Tous les Fichiers

```
# Document  
cat encrypted_files/document.txt  
  
# Rapport  
cat encrypted_files/rapport.txt
```

```
# Clients
cat encrypted_files/clients.txt
```

 **Question 5** : Tous les fichiers sont-ils lisibles ?

5. Documentation du Code

5.1 Créer un README pour l'Outil

Créez `TOOL_README.md` :

```
# Recovery Tool - Mode d'Emploi

## Installation

Pas de dépendances externes. Python 3.6+ requis.

## Utilisation

### Déchiffrer un fichier
```bash
python3 recovery_tool.py document.txt.encrypted
```

Déchiffrer un répertoire

```
python3 recovery_tool.py ./encrypted_files/
```

Comment ça marche ?

Algorithme

Limitations

---

### 6.2 Commenter Votre Code

Ajoutez des commentaires clairs.

---

## 7. Tests Avancés

### 7.1 Test de Robustesse

```
Fichier inexistant
python3 recovery_tool.py inexistant.txt.encrypted
```

```
Répertoire vide
mkdir empty_dir
python3 recovery_tool.py empty_dir/

Fichier déjà déchiffré
python3 recovery_tool.py encrypted_files/document.txt
```

 **Question 6** : Le script gère-t-il correctement les erreurs ?

---

## 7.2 Test de Performance

```
Créer un gros fichier
dd if=/dev/urandom of=big_file.bin bs=1M count=10

Le chiffrer (avec le ransomware ou manuellement)
Le déchiffrer et mesurer le temps
time python3 recovery_tool.py big_file.bin.encrypted
```

---

## 8. Comparaison avec l'Original

### 8.1 Vérifier avec **diff**

Si vous aviez les fichiers originaux :

```
diff document_original.txt document_decrypted.txt
```

Aucune différence = succès !

---

### 8.2 Vérifier avec des Hashes

```
Hash du fichier déchiffré
md5sum encrypted_files/document.txt

Comparer avec le hash du fichier original (si disponible)
```

---



## Livrables de l'Étape 4

À remettre :

-  Script **recovery\_tool.py** complété et testé

-  Les 3 fichiers déchiffrés (prouvés lisibles)
  -  `TOOL_README.md` avec mode d'emploi
  -  Réponses aux 6 questions
  -  (Optionnel) Améliorations du script
- 

## Prochaine Étape

Maintenant que vous pouvez récupérer les fichiers, passez à **l'Étape 5 : Pentest Web** pour analyser les vulnérabilités du serveur C2 !

---

**Bon recouvrement !**  